AUSTCAIC — AI & ML WING

Class 2: Exploratory Data Analysis & Data Preprocessing

Session Date: 25th July, 2025 • Time: 9:00 PM

OVERVIEW

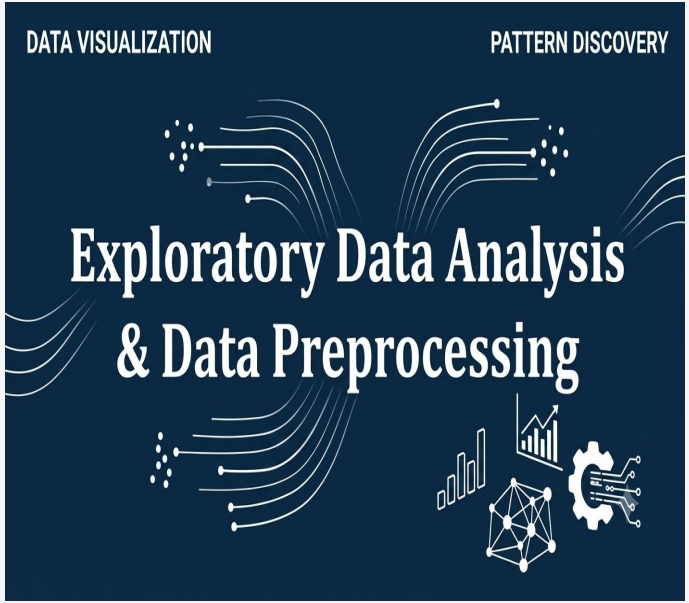
What We'll Cover Today

1. Why Data Preprocessing Matters
2. Features vs Target
3. Exploratory Data Analysis (EDA)
4. Data Visualization
5. Missing Values & Duplicates
6. Feature Scaling & Encoding
7. Intro to Machine Learning
8. How Data Powers ML

RECAP

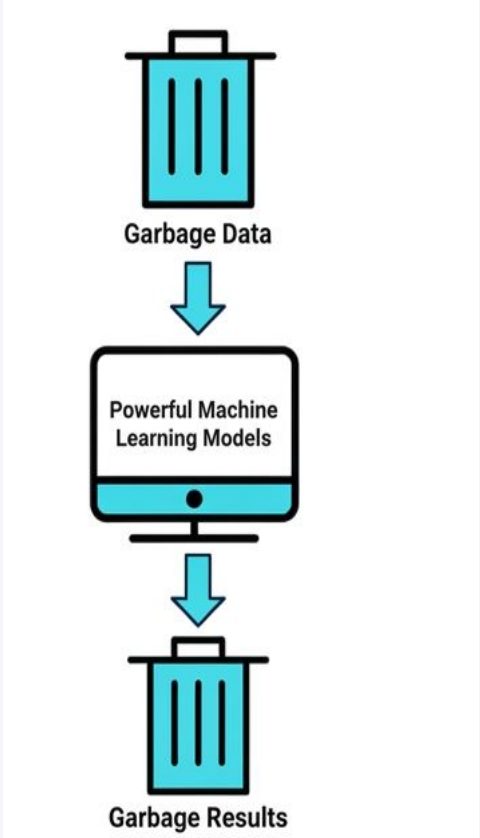
Quick Recap: Class 1

- We covered core Python: variables, data types, operators, conditionals, loops, functions.
- We covered data structures: lists, tuples, dictionaries, sets.
- built a simple calculator using input(), conditionals, and a loop.
- **Today we shift focus: how do we prepare raw data before it ever reaches a machine learning model?**



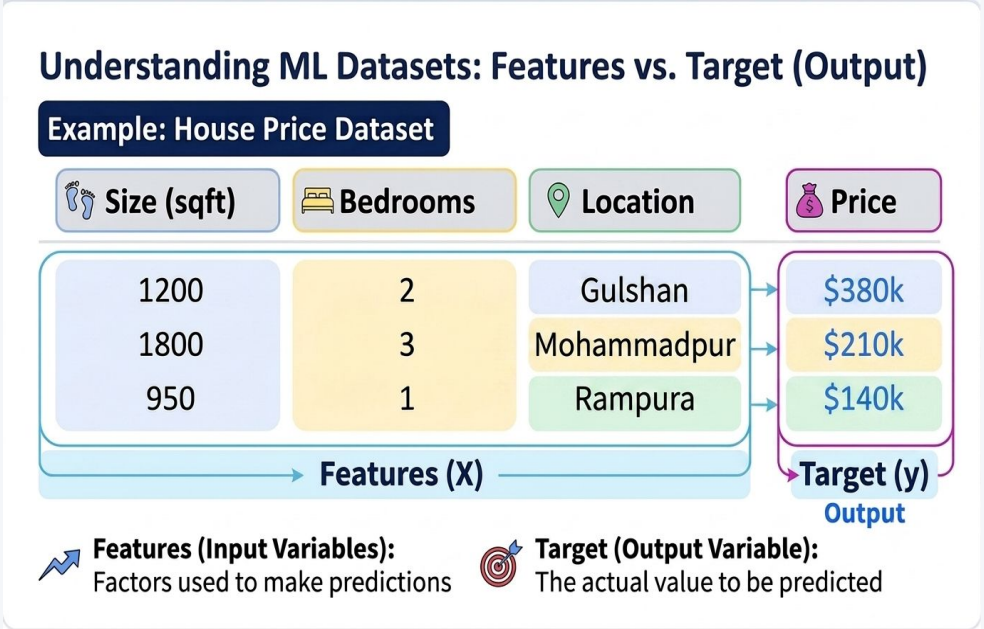
Why Data Preprocessing Is Important

- Real-world data is messy: missing values, duplicates, wrong formats, inconsistent scales.
- Models learn patterns directly from data — bad data produces bad models.
- **"Garbage In, Garbage Out" (GIGO): a model is only as good as the data it is trained on.**
- Preprocessing often takes 60–80% of a real data science project's time.
- Clean, well-structured data leads to faster training and more reliable predictions.



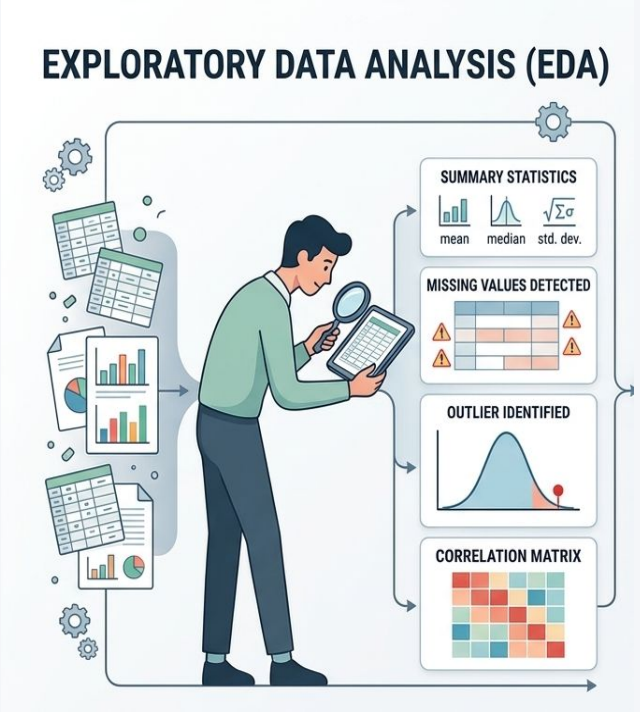
Understanding Datasets

- **Features (X):**
- The input variables used to make a prediction — also called independent variables.
- **Target (y):**
- The value we are trying to predict — also called the dependent variable or label.
- A dataset is typically organized as rows (samples) and columns (features + target).



What Is Exploratory Data Analysis?

- EDA is the process of investigating a dataset to summarize its main characteristics.
- Goal: understand the shape, structure, and quality of data before modeling.
- Answers questions like: What does the data look like? Are there patterns, outliers, or missing values?
- Combines statistics (mean, median, std) with visualizations (charts, plots).



A Typical EDA Workflow

1. **Load the data**
   Read the dataset into a Pandas DataFrame (df.read_csv, etc.)

2. **Inspect structure**
   df.head(), df.info(), df.shape, df.columns

3. **Summary statistics**
   df.describe() — mean, std, min, max, quartiles

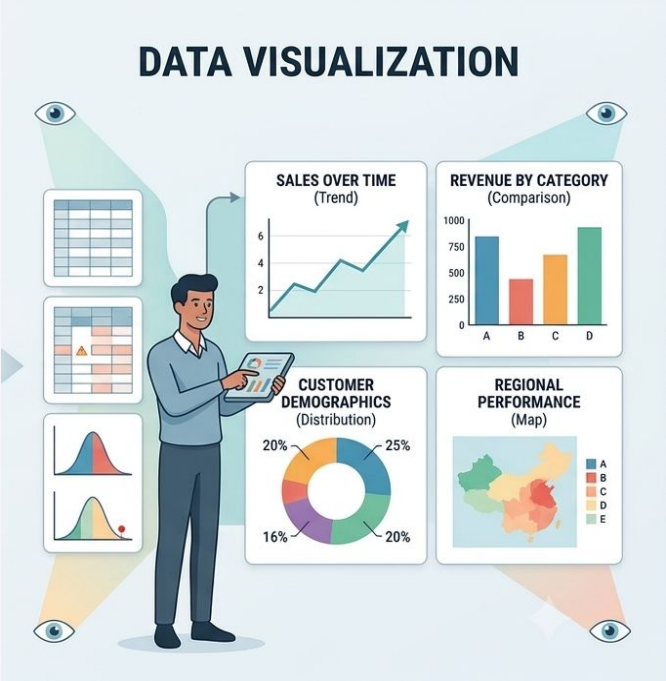
4. **Check data quality**
   Look for missing values, duplicates

5. **Visualize**
   Histograms, scatter plots, box plots, correlation heatmaps

6. **Draw insights**
   Note patterns, outliers, and relationships to guide preprocessing

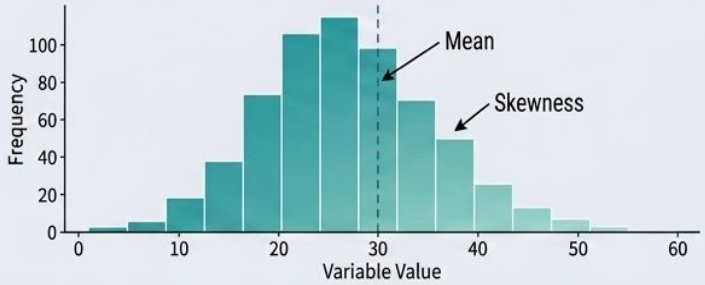
Why We Visualize Data

- Humans understand pictures faster than tables of numbers.
- Visualization reveals patterns, trends, and outliers that summary stats can miss.
- Helps communicate findings clearly to both technical and non-technical audiences.
- Guides decisions on cleaning, scaling, and feature engineering.

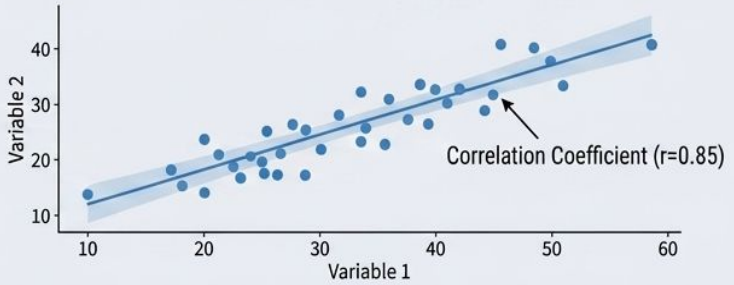


Common Plot Types

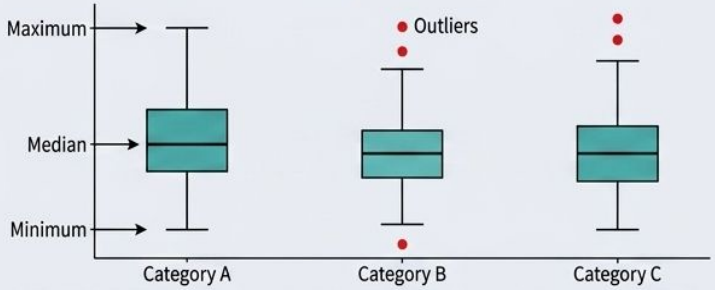
Histogram — Shows distribution of a single variable



Scatter Plot — Shows relationship between two variables



Box Plot — Shows spread, median, and outliers



Heatmap — Shows correlation between multiple features



Plotting with Matplotlib

- Matplotlib is the foundational plotting library in Python.
- pyplot gives a simple, MATLAB-style interface for quick charts.
- Almost every other Python plotting library builds on top of it.

```
import matplotlib.pyplot as plt

plt.hist(df['age'], bins=10)
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Count')
plt.show()
```

Plotting with Seaborn

- Seaborn is built on top of Matplotlib, with better defaults.
- Great for statistical plots: distributions, correlations, categories.
- Works seamlessly with Pandas DataFrames.

```
import seaborn as sns

sns.boxplot(x='class', y='fare',
            data=df)

sns.heatmap(df.corr(),
            annot=True, cmap='Blues')
```

Handling Missing Values

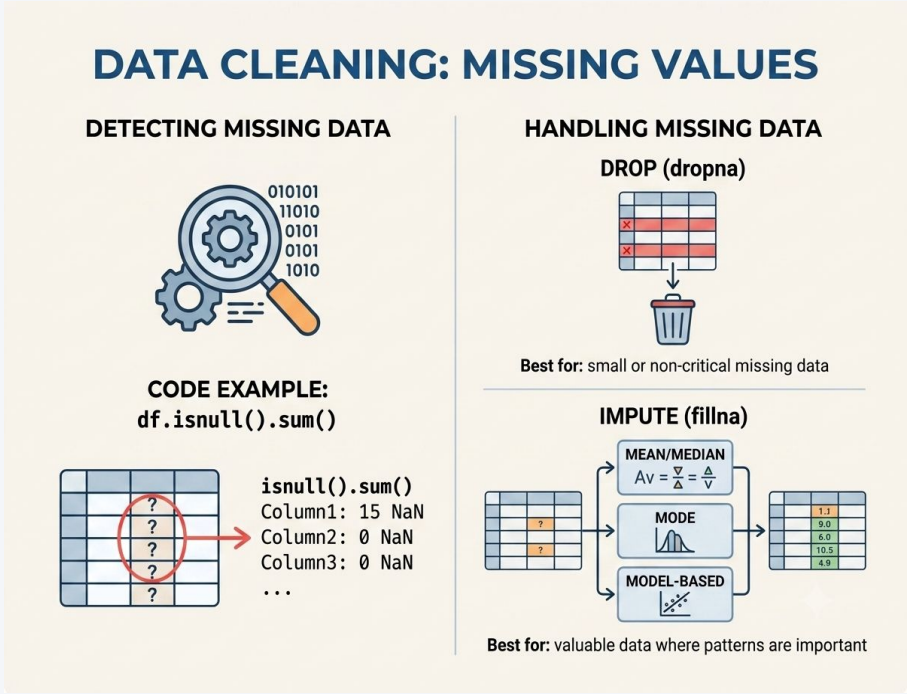
- Missing data is common: sensors fail, users skip form fields, records get merged imperfectly.

Detect:

- df.isnull().sum() shows how many values are missing per column.

Handle:

- Drop rows/columns (dropna) when missing data is small or non-critical.
- Impute (fillna) with mean, median, mode, or a model-based estimate when data is valuable.



Missing Values in Pandas

```
# Check missing values
df.isnull().sum()

# Drop rows with any missing value
df_clean = df.dropna()

# Fill missing values with column mean
df['age'] = df['age'].fillna(
    df['age'].mean()
)
```

Handling Duplicate Records

- Duplicate rows can skew statistics and bias a model toward repeated examples.
- Common causes: merging datasets, repeated form submissions, logging errors.
- df.duplicated() flags duplicate rows.
- df.drop_duplicates() removes them, keeping the first occurrence by default.

Removing Duplicates in Pandas

```
# Check for duplicate rows
df.duplicated().sum()

# Remove duplicate rows
df = df.drop_duplicates()

# Reset index after cleaning
df = df.reset_index(drop=True)
```

Feature Scaling

Many ML algorithms are sensitive to the scale of features — a column ranging 0–1 and one ranging 0–100,000 can unfairly dominate a model.

Normalization (Min-Max)

- Rescales values to a fixed range, usually 0 to 1
- Formula: (x - min) / (max - min)
- Sensitive to outliers

Standardization (Z-score)

- Rescales data to have mean = 0 and std = 1
- Formula: (x - mean) / std
- More robust to outliers than min-max

Scaling with Scikit-Learn

```
from sklearn.preprocessing import (
    MinMaxScaler, StandardScaler
)

scaler = StandardScaler()
df[['age','income']] = scaler.fit_transform(
    df[['age','income']])
```

Encoding Categorical Data

Machine learning models need numbers — categorical (text) columns must be converted before training.

Label Encoding

- Assigns each category an integer (0, 1, 2, ...)
- Compact, but implies an order that may not exist
- Good for ordinal data (e.g. Low/Medium/High)

One-Hot Encoding

- Creates a separate binary column per category
- No false ordering between categories
- Best for nominal data (e.g. City, Color)

Encoding in Pandas & Scikit-Learn

```
# One-hot encoding with Pandas
df = pd.get_dummies(df, columns=['city'])

# Label encoding with Scikit-Learn
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['grade'] = le.fit_transform(df['grade'])
```

A Brief Introduction to ML

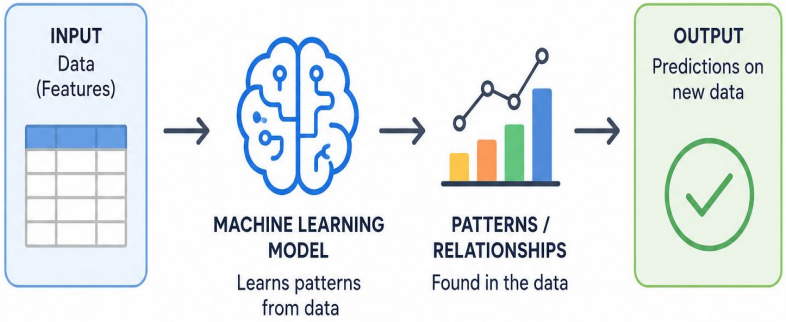
What Is Machine Learning?

Definition: A way for computers to learn patterns from data and make decisions or predictions — without being explicitly programmed with rules for every case.

Purpose: Instead of writing step-by-step instructions, we show the algorithm examples, and it figures out the underlying pattern itself.
How it works (in short):

- Feed the model data (features).
- The model finds patterns/relationships in that data.
- It uses those patterns to make predictions on new, unseen data.

Example: Show a model thousands of houses with their size, location, and price — it learns the relationship and can then predict the price of a new house.



How Data Powers Machine Learning



Everything covered today — cleaning, scaling, encoding — feeds directly into the "Preprocessing" stage. Better input data means a better-performing model on the other end.

SUMMARY

Key Takeaways

- Preprocessing determines the quality ceiling of any ML model — clean data first.
- EDA helps you understand your dataset before you touch a model.
- Visualization turns numbers into insight — use the right plot for the question.
- Handle missing values and duplicates deliberately, not by accident.
- Scale numeric features and encode categorical ones before training.